

REVIVAL SERIES

TRS MODE CARRIÈRE

Brief Technique — À l'attention de Boule

Rédigé par
VirtuAdem (IA Backend)

Destinataire
Boule — Dev Backend

Version
Phase 1 — MVP

Statut
En attente de validation

1. CONTEXTE DU PROJET

Le projet TRS Mode Carrière est une application web destinée à la communauté Revival Series sur Assetto Corsa. Elle permet aux pilotes de gérer une progression entre les courses : budget, achat et gestion de voitures, upgrades, inscriptions aux événements et consultation des résultats.

Ce document décrit l'architecture technique retenue pour la Phase 1 (MVP), les technologies choisies, et les informations dont nous avons besoin de ta part pour démarrer le développement.

2. ARCHITECTURE RETENUE

2.1 Vue d'ensemble

Tout est hébergé sur le même serveur Linux qui fait tourner ACSM. C'est la configuration la plus simple et la plus performante — pas de latence réseau entre les composants.

Composant	Technologie	Rôle
Frontend	HTML / CSS / JS	Interface pilote — servi par Flask
Backend	Python Flask	API REST + logique métier
Base de données	SQLite (évolutif)	Stockage données pilotes/courses
Auth	Discord OAuth2	Connexion pilote via compte Discord
Intégration ACSM	Accès fichier local	Dépôt entry_list.ini et setup fixes

2.2 Schéma de communication

```
graph TD
    NP["[ Navigateur Pilote ]"] -- HTTPS --> FB["[ Flask Backend - API REST ]"]
    FB -- SQLite --> BDD["[ BDD ]"]
    FB -- "Fichiers ACSM" --> ACSM["[ /acsm/events/... ]"]
    FB -- "Discord OAuth2" --> AD["[ API Discord ]"]
```

3. PÉRIMÈTRE PHASE 1 — MVP

La Phase 1 se concentre sur deux fonctionnalités core et leur infrastructure. Toute la suite du projet (garage, upgrades, résultats) sera construite sur ces fondations.

3.1 Fonctionnalité A — Inscription à un événement

Un pilote connecté via Discord peut consulter le calendrier des événements et s'inscrire. L'inscription déclenche automatiquement la génération et le dépôt de l'`entry_list.ini` sur le serveur ACSM.

- ① Pilote se connecte via Discord OAuth
- ② Flask crée ou met à jour son profil pilote en BDD (Discord ID, username, avatar)
- ③ Pilote sélectionne un événement et clique Inscription
- ④ Flask enregistre l'inscription en BDD (table Inscriptions)
- ⑤ Flask génère l'`entry_list.ini` depuis les données BDD et le dépose dans le dossier ACSM correspondant à l'événement

3.2 Fonctionnalité B — Upload de livrée

Le pilote peut uploader ses fichiers de skin directement depuis le site. Le backend valide les fichiers et les copie dans le dossier ACSM de la voiture correspondante. L'`entry_list` est mise à jour avec le bon nom de skin.

- Formats acceptés : fichiers `.dds` (textures) + `livery.png`
- Validation : taille, format, nommage
- Convention de nommage skin : `IDdiscord_IDcourse`
- Dépôt dans : `/acsm/content/cars/[IDfolder]/skins/[nom_skin]/`
- Mise à jour automatique du champ `SKIN=` dans l'`entry_list`

4. CONTRATS D'API — ENDPOINTS FLASK

Le frontend consomme ces endpoints REST. Toutes les réponses sont en JSON.

Méthode	Route	Auth requise	Description
GET	/auth/discord	Non	Redirige vers Discord OAuth
GET	/auth/callback	Non	Callback OAuth — crée session
GET	/api/me	Oui	Profil du pilote connecté
GET	/api/events	Non	Liste des événements à venir
POST	/api/events/:id/register	Oui	Inscription + génération entry_list
DELETE	/api/events/:id/register	Oui	Désinscription + MAJ entry_list
POST	/api/livery/upload	Oui	Upload skin + dépôt ACSM
GET	/api/admin/events	Admin	Gestion événements (admin)
POST	/api/admin/events	Admin	Créer un événement
POST	/api/admin/entry-list/:id	Admin	Régénération manuelle entry_list

5. BASE DE DONNÉES — TABLES PHASE 1

SQLite pour la Phase 1. Structure évolutive vers PostgreSQL. Voici les tables nécessaires au MVP :

Table : Pilote

Champ	Type	Description
IDpilote	INTEGER PK	Identifiant interne auto-increment
DiscordID	TEXT UNIQUE	ID Discord (clé d'authentification)
Username	TEXT	Pseudo Discord
SteamID	TEXT	Steam ID (renseigné par le pilote)
Budget	REAL	Solde en monnaie virtuelle (€)
DateArrivee	TEXT	Date d'inscription
DerniereConnexion	TEXT	Timestamp dernière connexion
IDequipe	INTEGER FK	Référence table Equipe
IDrole	INTEGER FK	Référence table Role (pilote/admin)

Table : Courses

Champ	Type	Description
IDcourse	INTEGER PK	Identifiant interne
Nom	TEXT	Nom de l'événement
Circuit	TEXT	Nom du circuit

Date	TEXT	Date et heure de la course
NbSlots	INTEGER	Nombre de places disponibles
IDchamp	INTEGER FK	Championnat associé
IDeventACSM	TEXT	Identifiant côté ACSM (pour le chemin fichier)
Realisee	INTEGER	0 = à venir / 1 = terminée

Table : Inscriptions

Champ	Type	Description
IDcourse	INTEGER FK	Référence course
IDpilote	INTEGER FK	Référence pilote
IDvoiture	INTEGER FK	Référence voiture (garage) — null Phase 1
IDmodele	INTEGER FK	Référence modèle voiture
IDserie	INTEGER FK	Catégorie de course
IDfolder	TEXT	Dossier AC de la voiture
NomSkin	TEXT	Nom du dossier skin upladé

6. INTÉGRATION ACSM

6.1 Génération entry_list.ini

À chaque inscription/désinscription, Flask régénère le fichier entry_list.ini complet pour la course concernée et le dépose directement dans le dossier ACSM (accès fichier local, pas de SSH nécessaire).

```
[CAR_0]
DRIVERNAME=Nom pilote
TEAM=Nom équipe
MODEL=IDfolder
SKIN=IDdiscord_IDcourse
GUID=SteamID
SPECTATOR_MODE=0
BALLAST=0
RESTRICTOR=0
FIXED_SETUP=
```

Note : BALLAST et FIXED_SETUP seront vides en Phase 1. Ils seront calculés et remplis dès l'activation du module Garage/Upgrades.

6.2 Structure de dossiers ACSM attendue

Pour que Flask puisse déposer les fichiers au bon endroit, nous avons besoin de connaître la structure exacte. Exemple probable :

```

/[racine_acsm]/
  events/
    [IDeventACSM]/
      entry_list.ini ← Flask écrit ici
content/cars/[IDfolder]/
  skins/[IDdiscord_IDcourse]/
    livery.png ← Flask copie ici
    preview.jpg
    *.dds

```

7. INFORMATIONS NÉCESSAIRES — ACTION REQUISE

Pour démarrer le développement, voici les informations dont VirtuAdem a besoin. Ce sont des blockers — sans elles, on ne peut pas avancer.

#	Information	Pourquoi c'est nécessaire	Statut
1	Chemin racine ACSM sur le serveur	Pour que Flask sache où déposer les fichiers entry_list et skins	⚠ Manquant
2	Structure exacte dossier events ACSM	Nommage des dossiers par événement (UUID ? nom ? numéro ?)	⚠ Manquant
3	Version ACSM installée	Vérifier compatibilité API si on veut l'utiliser plus tard	⚠ Manquant
4	Création app Discord Developer	Récupérer CLIENT_ID et CLIENT_SECRET pour OAuth	⚠ À faire ensemble
5	URL publique du serveur (IP ou domaine)	Pour configurer l'URL de callback OAuth Discord	⚠ Manquant
6	Port disponible sur le serveur	Flask a besoin d'un port ouvert (défaut 5000)	⚠ À confirmer

8. PLAN DE DÉVELOPPEMENT

Sprint	Durée estimée	Contenu	Livrable
Sprint 1	1-2 jours	Structure Flask, BDD SQLite, OAuth Discord complet	Login Discord fonctionnel
Sprint 2	2-3 jours	Page Calendrier, inscription événement, génération entry_list.ini + dépôt ACSM	Inscription opérationnelle
Sprint 3	2-3 jours	Upload livrée, validation fichiers, copie dossier ACSM, MAJ entry_list	Livrées personnalisées
Sprint 4	1-2 jours	Interface admin : créer événement, voir inscriptions, régénération manuelle entry_list	Back-office minimal

Le frontend de chaque sprint sera du HTML fonctionnel avec classes CSS sémantiques. Lokopixo peut habiller chaque module indépendamment sans toucher au JS.

9. QUESTIONS OUVERTES

Ces points nécessitent une décision avant ou pendant le développement :

- Gestion de la livrée par défaut : si un pilote ne upload pas de skin, quelle skin est assignée dans l'entry_list ? (skin par défaut du modèle ?)
- SteamID obligatoire à l'inscription : est-ce qu'on bloque l'inscription si le pilote n'a pas renseigné son Steam ID, ou on laisse vide pour l'instant ?
- Droits fichiers ACSM : l'utilisateur qui fait tourner Flask a-t-il les droits d'écriture dans les dossiers ACSM ?
- Coexistence avec un bot Discord de livrées : si un bot tiers gère déjà les skins, on prévoit une convention de nommage commune ou on désactive l'upload via le site ?